01

BASES
DE DONNÉES
MODERNES

Apprendre
Comprendre
Construire demain

CHAPITRE 3

Neo4j
Bases de données orientées graphe

Le projet : la base de données d'un restaurant

Ce que nous allons construire dans ce chapitre

1 Les clients

Ils commandent, notent les plats et parrainent leurs proches.

2 Les commandes

Chaque passage d'un client : la date et les plats commandés.

3 Les plats

La carte du restaurant : le nom, le prix et la catégorie.

4 Les ingrédients

Ce qui compose chaque plat, avec les allergènes signalés.

i **Toute la base sera créée directement dans Neo4j, sans passer par une base relationnelle.**

Les données à enregistrer

Quatre entités, chacune avec ses propriétés

Entité	Ce que l'on enregistre	Exemple
Client	identifiant, nom, ville	Sara, Casablanca
Commande	numéro, date	n° 101, le 12 octobre
Plat	nom, prix, catégorie	Tajine de poulet, 85 DH
Ingrédient	nom, allergène (oui / non)	Amandes, oui

i Les données comptent, mais l'essentiel du projet est ailleurs : dans les liens entre elles.

L'objectif : répondre aux questions du gérant

Chaque question revient à suivre des liens

Ce que le gérant veut savoir	Le chemin à suivre	Pas
1. Quels plats Sara a-t-elle commandés ?	Client → Commande → Plat	2
2. Quels plats contiennent un allergène ?	Plat → Ingrédient	1
3. Qui Sara a-t-elle fait venir, même indirectement ?	Client → Client → …	Inconnu
4. Quel plat proposer à Lina ?	Lina → ses plats → clients proches → leurs plats	6

i Objectif : une base où chaque réponse se trouve en suivant les liens, sans accumuler les jointures.

Représenter les données : un graphe

Quatre briques suffisent pour tout représenter



N Nœud

Un élément : un client, un plat...
`(sara)`
Sur le dessin : un rond.

R Relation

Un lien orienté entre deux nœuds.
`-[:A_PASSE]->`
Nom en MAJUSCULES.

L Label

La catégorie d'un nœud.
`(:Client)  (:Plat)`
Nom en PascalCase, au singulier.

P Propriété

Une clé-valeur, sur un nœud ou une relation.
`{nom: 'Tajine', prix: 85}`
Nom en camelCase.

Neo4j, SQL et MongoDB : les équivalences

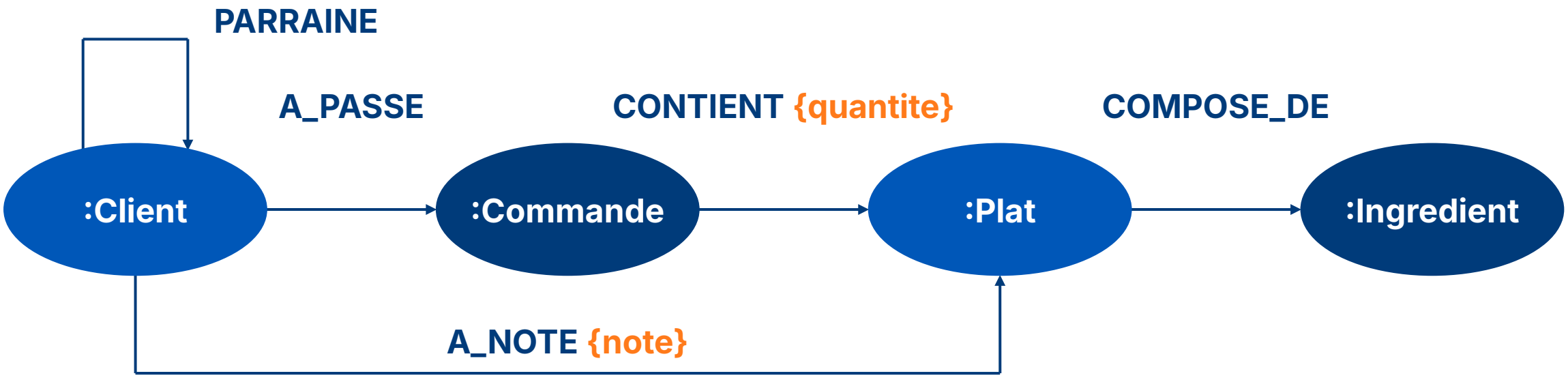
Les mêmes idées, sous d'autres noms

Notion	SQL (MySQL)	MongoDB	Neo4j
Un élément	une ligne	un document	**un nœud**
Sa catégorie	une table	une collection	**un label**
Une information	une colonne	un champ	**une propriété**
Un lien	clé étrangère + jointure	imbrication ou $lookup	**une relation**
Le schéma	fixe	souple	**souple**

i | En Neo4j, le lien est enregistré une fois pour toutes : on le suit, on ne le recalcule pas.

Le modèle du restaurant en graphe

4 labels et 5 types de relation



i | Des propriétés sur les nœuds (nom, prix, ville) comme sur les relations (quantite, note).

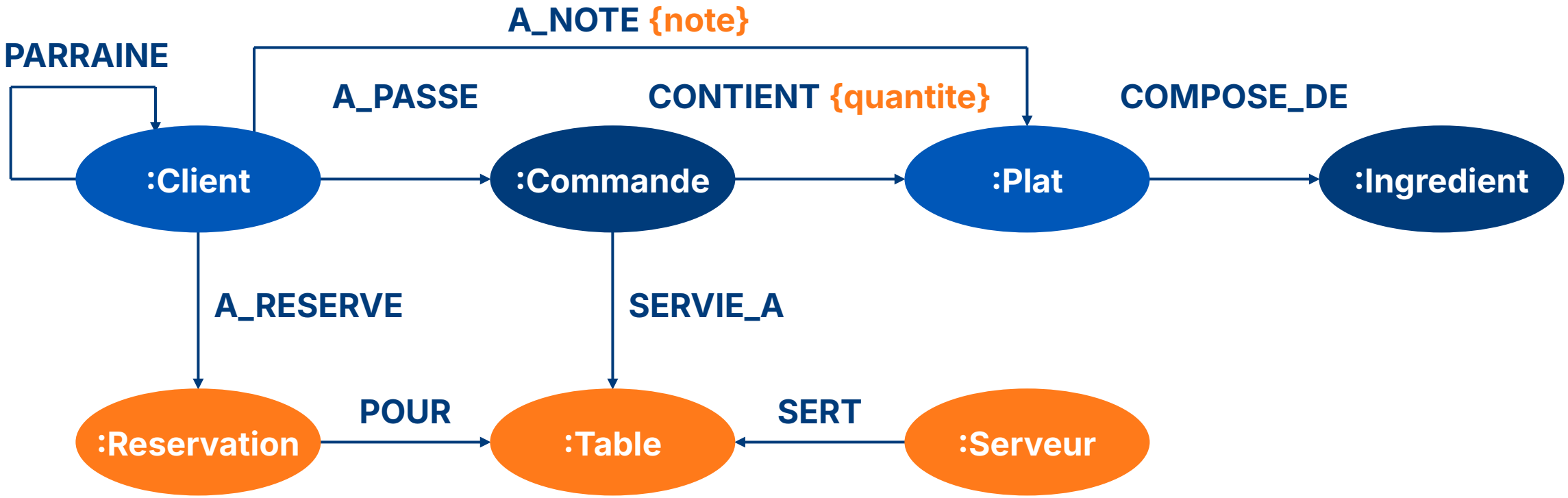
Activité : compléter le modèle

Le restaurant veut aussi gérer ses tables, ses réservations et ses serveurs

1. Quels nouveaux nœuds faut-il créer ? Donnez leurs labels.
2. Quelles relations les relient au modèle existant ? Précisez le sens.
3. Quelles propriétés, et où : sur un nœud ou sur une relation ?
4. Dessinez le graphe complet du restaurant.

Corrigé : le modèle complété

3 nouveaux labels (en orange) et 4 nouvelles relations



Du dessin à Cypher

On écrit ce que l'on dessine

Sur le dessin	En Cypher
Un rond : un nœud	`( )`
Son label et ses propriétés	`(:Client {nom: 'Sara'})`
Une flèche : une relation	`-[:PARRAINE]->`
Un nombre de pas inconnu	`-[:PARRAINE*]->`

i | Un pas de plus : une flèche de plus. Aucune jointure.

Créer : CREATE

On écrit le dessin, Neo4j le crée

```
// Créer des nœuds
CREATE (:Client {nom: 'Sara', ville: 'Casablanca'});
CREATE (:Plat {nom: 'Tajine de poulet', prix: 85});

// Créer deux nœuds et leur relation, en une seule requête
CREATE (:Client {nom: 'Sara'})-[:PARRAINE]->(:Client {nom: 'Nadia'});

// Avec des variables (s, r, n) pour afficher ce qui vient d'être créé
CREATE (s:Client {nom: 'Sara', ville: 'Tanger'})-[r:PARRAINE]->
       (n:Client {nom: 'Nadia', ville: 'Tanger'})
RETURN s, r, n;
```

i **Attention :** CREATE crée toujours. Sara existait déjà : elle est maintenant en double.

Créer sans doublon : MERGE

MERGE crée seulement si l'élément n'existe pas encore

```
// MERGE cherche d'abord, et crée seulement si rien n'est trouvé
MERGE (:Client {nom: 'Sara'});    // existe déjà : rien n'est créé
MERGE (:Client {nom: 'Karim'});   // n'existe pas : Karim est créé

// Une relation : un MERGE par nœud, puis un pour la relation
MERGE (a:Client {nom: 'Sara'})    // a et b : des variables
MERGE (b:Client {nom: 'Karim'})
MERGE (a)-[:PARRAINE]->(b);
```

Piège : MERGE cherche le motif entier. Écrit en une seule ligne, il peut recréer Sara en double.

Afficher : MATCH et RETURN

MATCH retrouve, RETURN affiche

```
// Choisir l'entité à afficher grâce au label : vue graphe
MATCH (c:Client) RETURN c;      // seulement les clients
MATCH (p:Plat) RETURN p;        // seulement les plats
MATCH (n) RETURN n;             // tout le graphe, sans filtre

// Seulement le nom et la ville des clients : vue tableau
MATCH (c:Client)
RETURN c.nom, c.ville;

// Les clients parrainés par Sara
MATCH (s:Client {nom: 'Sara'})-[:PARRAINE]->(f:Client) RETURN f.nom;
```

i **À retenir :** MATCH ne crée jamais rien, il retrouve ce qui existe déjà.

Relier deux nœuds existants

MATCH retrouve les nœuds, CREATE trace la relation

```
// Sara et Nadia existent : on les retrouve, puis on les relie
MATCH (a:Client {nom: 'Sara'}), (b:Client {nom: 'Nadia'})
CREATE (a)-[:PARRAINE]->(b);

// Avec MERGE : crée le nœud s'il est introuvable
MERGE (a:Client {nom: 'Sara'})
MERGE (b:Client {nom: 'Nadia'})
MERGE (a)-[:PARRAINE]->(b);
```

i **Différence :** si un nom est mal écrit, MATCH ne crée rien ; MERGE crée un nœud en trop.

Structure d’une requête

On dessine, on filtre, on retourne

Le squelette

```
MATCH    (motif à trouver)
WHERE    conditions
RETURN   ce que l'on veut voir
ORDER BY tri
SKIP n LIMIT m
```

SQL	Cypher
FROM + JOIN	MATCH
WHERE	WHERE
SELECT	RETURN (à la fin)
LIMIT, OFFSET	LIMIT, SKIP

i On décrit d'abord le motif, puis on choisit ce que l'on garde : RETURN se place à la fin.

Filtrer : WHERE et ORDER BY

Ne garder que ce qui nous intéresse

```
// Les plats à moins de 90 DH, du plus cher au moins cher
MATCH (p:Plat)
WHERE p.prix < 90
RETURN p.nom, p.prix ORDER BY p.prix DESC;

// Les clients de Casablanca ou de Rabat
MATCH (c:Client)
WHERE c.ville IN ['Casablanca', 'Rabat']
RETURN c.nom;
```

i | **Opérateurs :** `=` `<>` `<` `>` `AND` `OR` `NOT` `IN` `CONTAINS` `IS NULL`

Afficher en suivant les relations

Chaque flèche remplace une jointure

```
// Les plats commandés par Sara : deux pas, zéro jointure
MATCH (:Client {nom: 'Sara'})-[:A_PASSE]->(:Commande)
      -[:CONTIENT]->(p:Plat)
RETURN DISTINCT p.nom;
// Tajine de poulet, Salade marocaine, Pastilla au poisson

// Filtrer sur une propriété de relation
MATCH (c:Client)-[n:A_NOTE]->(p:Plat)
WHERE n.note >= 4
RETURN c.nom, p.nom, n.note;
```

i | Sans pointe de flèche, `-[:PARRAINE]-` suit le lien dans les deux sens.

Modifier et supprimer

SET, REMOVE, DELETE

```
// Modifier une propriété
MATCH (p:Plat {nom: 'Salade marocaine'}) SET p.prix = 38;

// Ajouter un label, retirer une propriété
MATCH (c:Client {nom: 'Sara'}) SET c:Fidele REMOVE c.email;

// Supprimer un nœud ET toutes ses relations
MATCH (c:Client {nom: 'Nadia'}) DETACH DELETE c;
```

i **DELETE seul échoue** si le nœud a encore des relations : on utilise DETACH DELETE.